# Server-Side Template Injection to Remote Code Execution

**Vulnerability class:** Server-Side Template Injection (SSTI) leading to Remote Code Execution (RCE) **Target:** OWASP Juice Shop (lab instance, `192.168.100.51:3000`) **Affected component:** User profile page (`/profile`), username field **Template engine:** Pug (formerly Jade) **Author:** David Gorman **Date:** 31 October 2026

---

## 1. Introduction

This report documents a Server-Side Template Injection (SSTI) vulnerability that escalates to command execution, demonstrated against the OWASP Juice Shop user profile page. It covers how the vulnerability arises in the application's template handling, how it is identified and exploited, the impact and its limits, and how the attack can be detected. The work was carried out in a controlled, isolated lab, and the finding is presented as a proof of concept of the attack together with analysis of the underlying template-engine behaviour that causes it.

### 1.1 Scope

**In scope**

- The single OWASP Juice Shop instance running on the lab Ubuntu Server VM (`192.168.100.51:3000`).
- The authenticated user profile page at `/profile` and its username field, including the username update request that feeds it.
- Authenticated testing using a low-privilege account created through normal self-registration.
- Submission of any template injection payload, including full command-execution gadgets, to prove the concept. This is authorised because the lab is isolated on a host-only network and the application contains the payloads, so even a full RCE gadget cannot affect the host or any other system.

**Out of scope**

- Any host other than the designated Juice Shop VM, including the hypervisor, the Kali attack VM as a target, and anything beyond the isolated host-only network.
- The underlying Ubuntu operating system and any services on it other than the Juice Shop application.
- The public OWASP Juice Shop demo and any internet-facing instance.
- Social engineering and physical access.

### 1.2 Limitations

- **Proof of concept, not a full exploit chain.** The write-up demonstrates the SSTI to command-execution primitive using simple commands that return output, such as `id` and `whoami`. It proves code and command execution and stops there. It does not extend to an interactive shell, persistence, or a full post-exploitation chain. Attempts to escalate to a shell did not succeed in this environment (see Enumeration, Step 6).
- **Controlled lab, single target.** The technique is demonstrated against one deliberately vulnerable application in an isolated lab. It illustrates the SSTI to RCE vulnerability class rather than a production

exploitation campaign.

- **Engine and context specific.** SSTI exploitation depends heavily on the template engine in use and on how user input reaches the template. The injection syntax and escalation shown here apply to this engine (Pug) and this injection context, and would need adapting for other engines.
- **Guided identification.** Because the target is a gamified training application, the vulnerability class was known in advance (see the testing-context note in Section 3.2).

---

## 2. Executive Summary

The OWASP Juice Shop user profile page renders the logged-in user's username through the Pug server-side template engine. Because the username reaches the template in a way that causes it to be evaluated rather than treated as plain data, an attacker can submit a Pug interpolation expression that the server executes. Pug compiles to JavaScript, so the injected expression runs with access to the Node.js runtime, including the ability to load `child_process` and run operating-system commands.

Testing confirmed arbitrary command execution on the host as the `juice` service account. The payload `#{require('child_process').execSync("id")}` returned the account's real user and group identity, including membership of the `sudo` group. The attack requires only a low-privilege authenticated account and no victim interaction, since the attacker sets their own username and then loads their own profile.

Escalation to an interactive shell was not achieved in the lab: more invasive payloads caused the profile page to error and the account's profile to become temporarily unreachable. The finding is therefore a confirmed command-execution vulnerability demonstrated through discrete probing commands, not a full interactive compromise.

**Risk rating: Critical (CVSS 3.0 base score 9.9)** `CVSS:3.0/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:H`

The root cause is that user input is evaluated as template code rather than passed in as template data. The fix is to bind the username as a template variable, validate it on input, and remove the code-evaluation path.

---

## 3. Findings

### 3.1 Findings Summary

| ID | Title | Severity | CVSS | Status |
|---|---|---|---|---|
| F-01 | SSTI in profile username leads to command execution (RCE) | Critical | 9.9 | Confirmed |

### 3.2 Findings Details

#### F-01: Server-Side Template Injection to Remote Code Execution

| Field | Value |
|---|---|
| Severity | Critical |
| CVSS 3.0 | `9.9` (`CVSS:3.0/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:H`) |

| Field | Value |
|---|---|
| Location | `POST /profile` (payload delivery), `GET /profile` (trigger), username field |
| Source | `routes/userProfile.ts`, `routes/updateUserProfile.ts`, `views/userProfile.pug` |
| CWE | CWE-1336 (Improper Neutralization of Special Elements Used in a Template Engine), CWE-94 (Improper Control of Generation of Code) |
| Template engine | Pug (formerly Jade) |

**MITRE ATT&CK mapping**

| Tactic | Technique | ID |
|---|---|---|
| Initial Access | Exploit Public-Facing Application | T1190 |
| Execution | Command and Scripting Interpreter: JavaScript | T1059.007 |
| Execution | Command and Scripting Interpreter: Unix Shell | T1059.004 |
| Discovery | System Owner/User Discovery | T1033 |

**CVSS justification**

| Metric | Value | Rationale |
|---|---|---|
| Attack Vector | Network (N) | The profile endpoint is exposed over HTTP and exploited remotely. |
| Attack Complexity | Low (L) | No special conditions or timing. Setting the username and reloading the profile reliably triggers evaluation. |
| Privileges Required | Low (L) | A valid authenticated session is required. Registration is open, but the attacker must still hold a low-privilege account. |
| User Interaction | None (N) | No other user is involved. The attacker targets their own profile. |
| Scope | Changed (C) | The vulnerability is in the web application, but exploitation executes operating-system commands on the host as the `juice` account, affecting a component beyond the application's own security authority. |
| Confidentiality | High (H) | Command execution allows reading any file or secret accessible to the service account. |
| Integrity | High (H) | Arbitrary commands can modify files and data. |
| Availability | High (H) | The service can be disrupted or crashed. |

Base score: **9.9 (Critical)**. The Scope change is the decisive metric: SSTI that reaches operating-system command execution breaks out of the application's security authority and impacts the host, which is what separates this from an application-only compromise.

Temporal metrics can be added to reflect the testing maturity: `E:P` (Exploit Code Maturity: Proof-of-Concept), as a working PoC was demonstrated but not weaponised into a full exploit chain, and `RC:C` (Report Confidence: Confirmed), as the vulnerability was directly confirmed through successful command execution. The full vector `CVSS:3.0/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:H/E:P/RC:C` gives a temporal score of 9.4.

**Description and root cause**

The profile page is one of the few server-rendered pages in Juice Shop and uses the Pug template engine. The user's stored username is incorporated into the server-side Pug rendering in a way that causes Pug interpolation syntax ( `#{ }` ) in the username to be evaluated rather than displayed. Because Pug compiles to JavaScript, an interpolation expression has access to the Node.js runtime, including `require`, which allows loading `child_process` and executing operating-system commands.

The username is attacker-controlled. It is set through `POST /profile` (handled in `routes/updateUserProfile.ts` ), which stores the value with no validation of template metacharacters. No other user is involved: the attacker updates their own username and then loads their own profile, at which point the stored expression is evaluated. The root cause is that user input is treated as template code rather than template data.

**Enumeration and Identification**

*Testing context (disclaimer).* OWASP Juice Shop is a deliberately vulnerable, gamified training application. Because of this, the profile page was approached already knowing that Server-Side Template Injection was the vulnerability class of interest. In a real, blind engagement the tester would not have that foreknowledge, and classifying the vulnerability would be part of the work. The enumeration below documents a sound identification methodology, while acknowledging it was guided by the known nature of the target.

**Step 1: Passive technology fingerprinting.** The application was checked with Wappalyzer, which returned no technologies (Figure 1). This is worth noting for methodology: Wappalyzer fingerprints client-side frameworks and server banners, not the server-side template engine, so even a positive result would not have revealed Pug. Passive fingerprinting gave nothing to work with, so identification required active testing.

**Step 2: Page source review.** The profile page source was reviewed (Figure 2). Unlike most of the application, which is an Angular single-page app, the profile page is returned as server-rendered HTML with inline styles. Searching the source for keywords surfaced a reference to `beer.min.js`, confirming client-side JavaScript is in use. The more significant observation for this attack is that the profile page is server-rendered, which is a precondition for server-side template injection.

**Step 3: Benign baseline.** The username was set to a benign value ( `test` ) and the request captured in Burp Suite (Figure 3). The request was a standard form-encoded `POST /profile` with `username=test`, and the server responded `302 Found`, redirecting back to `/profile`. Nothing in the request or response revealed how the value was processed, so this served as a clean baseline for comparison.

**Step 4: Engine identification.** Two PortSwigger articles were read for background: the Web Security Academy page on SSTI and James Kettle's research paper. They explain how SSTI arises and name a number of commonly used template engines with example syntax, the research paper working through engines from FreeMarker down to Jade. That list of engines and their syntaxes was used as a checklist, submitting each one against the username field until a payload was evaluated. The `{{1+1}}` payload rendered literally, shown escaped as `\{{1+1}}` (Figure 4), ruling out the `{{ }}` family such as Twig and Jinja. The research paper's Jade entry (the Node.js engine, since renamed Pug) pointed to Pug's interpolation syntax: `#{1+1}` rendered as `2` (Figure 5), confirming

server-side evaluation and identifying the engine as Pug. The `#{ }` delimiter is Pug's interpolation syntax, while the expression inside it is evaluated as JavaScript, which is what makes command execution possible.

**Step 5: Confirming command execution.** Because Pug compiles to JavaScript, a Node command-execution payload was tested. `#{require('child_process').execSync("whoami")}` returned `juice` (Figure 6), and `#{require('child_process').execSync("id")}` returned `uid=1000(juice) gid=1000(juice)` with group membership including `sudo` (Figure 7). This confirmed arbitrary command execution on the host as the `juice` service account. The `sudo` group membership is notable, as it points to privilege-escalation potential beyond the initial execution context. These discovery commands map to ATT&CK T1033.

**Step 6: Attempting shell escalation.** Attempts were then made to move from single-command execution to an interactive shell, including base64-encoded and otherwise obfuscated payloads. These did not succeed: invasive payloads caused the profile page to return `HTTP 500`, and the account's profile became unreachable until the application was restarted. Command execution was demonstrable for discrete, output-returning commands, but was not escalated to an interactive session in this environment.

*Figures*

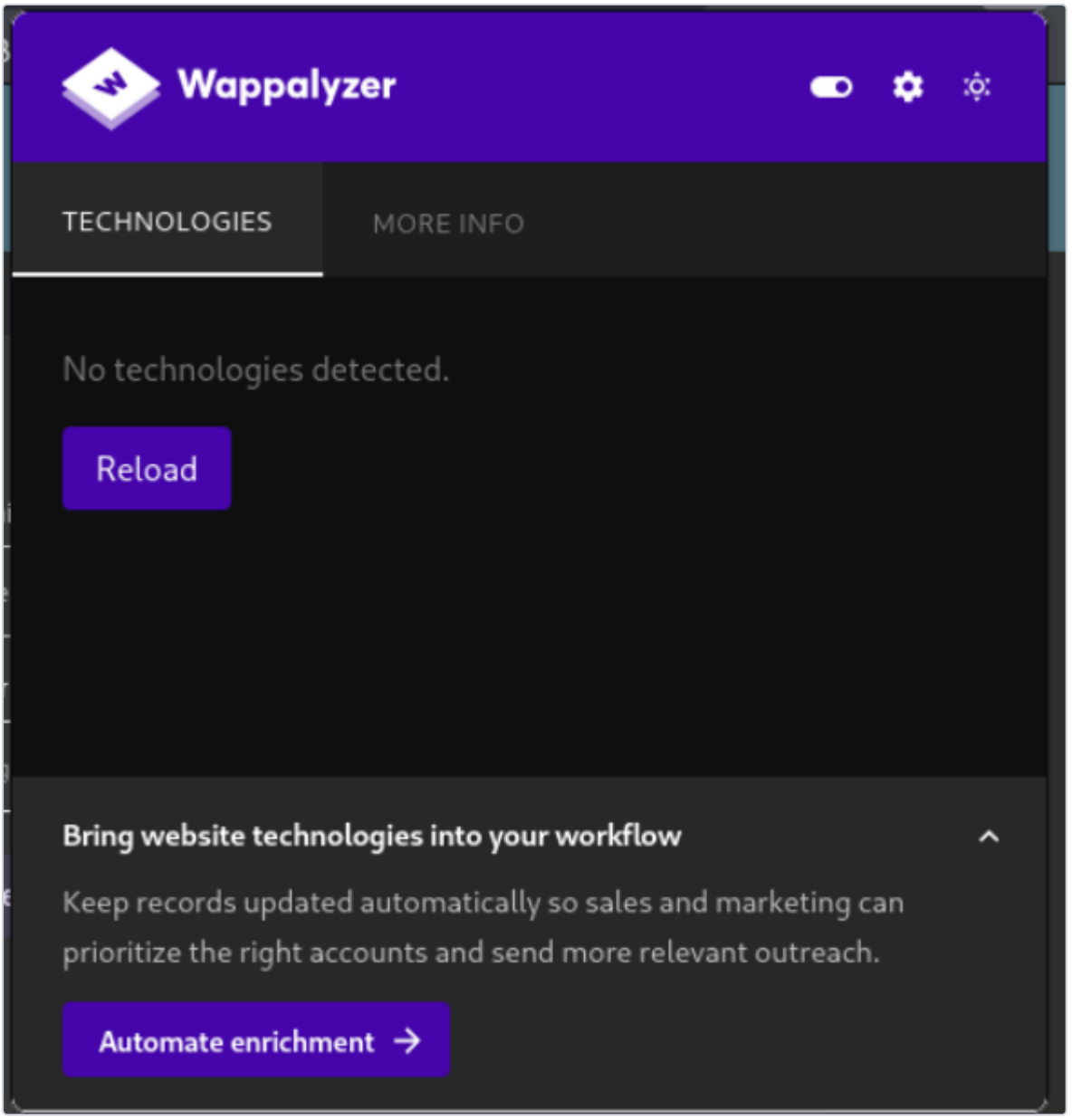



Figure 1. Wappalyzer, no technologies detected.

```
<!DOCTYPE html><html lang="en"><head><title>OWASP Juice Shop</title><meta charset="utf-8"><meta name="description" content=""><meta name="keywords" content=""><meta name="
article { background: #3e3e3e !important; }
button.fill { background-color: #4f6f7a !important; color: #FFFFFF !important; }
.profile-field label { color: #FFFFFF !important; font-size: 13px !important; display: block !important; margin-bottom: 4px; }
.profile-field input { border: 1px solid #FFFFFF !important; border-radius: 4px !important; padding: 12px !important; font-size: 14px !important; color: #FFFFFF !important
.profile-field input::placeholder { color: rgba(255,255,255,0.5); font-size: 0.85em; }
.profile-field { margin-bottom: 12px; }
.brand { display: flex; align-items: center; gap: 8px; text-decoration: none; color: #FFFFFF; }
.brand img { max-height: 50px; width: auto; }
.brand span { font: 500 20px/32px Roboto,"Helvetica Neue",sans-serif; }</style></head><body><header style="background: #4f6f7a; padding: 8px 16px;"><nav><a href="./#/" sty
```

Figure 2. Profile page server-rendered source, `beer.min.js` reference.

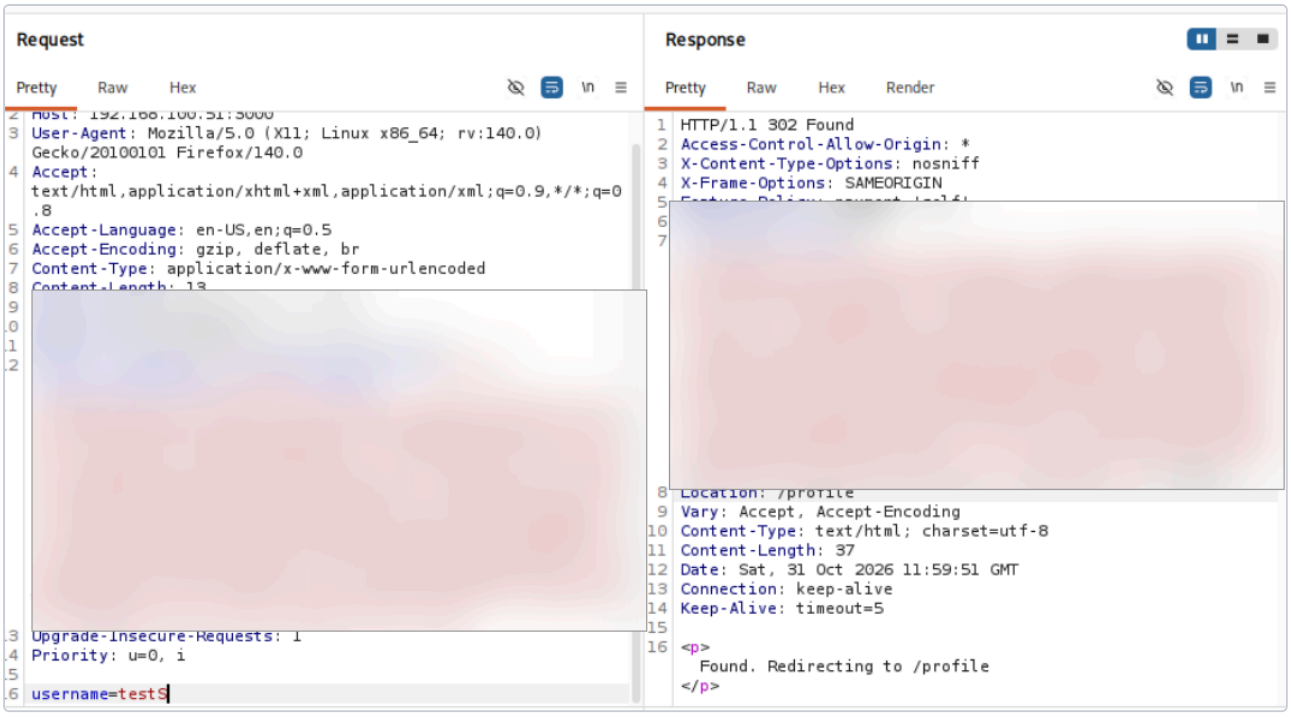



Figure 3. Burp capture, benign `username=test`, `302` response. Session token redacted.

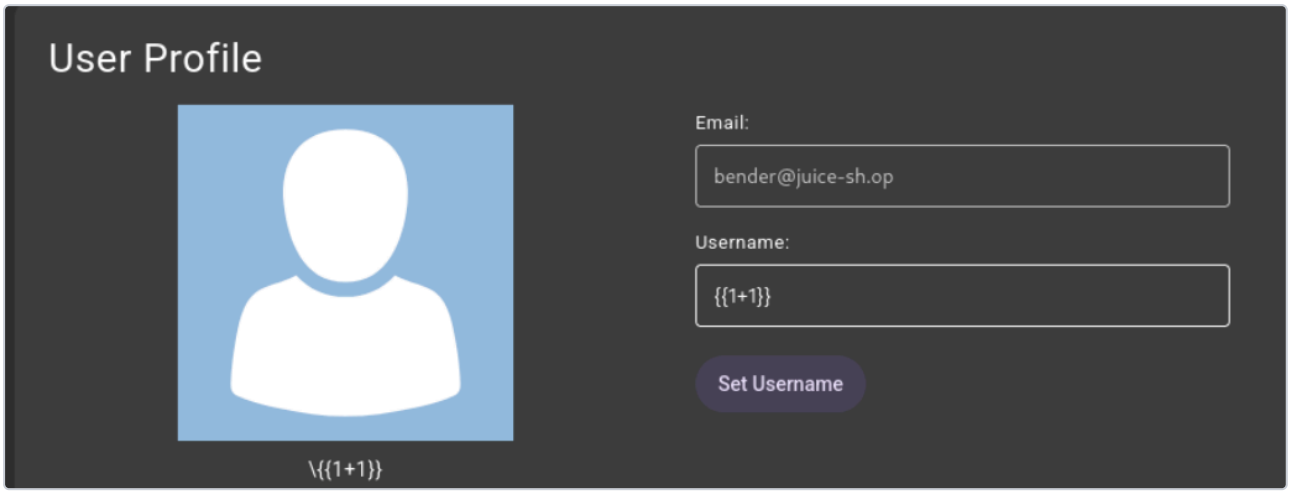



Figure 4. `{{1+1}}` rendered literally (`\{{1+1}}`), rules out `{{ }}` engines.

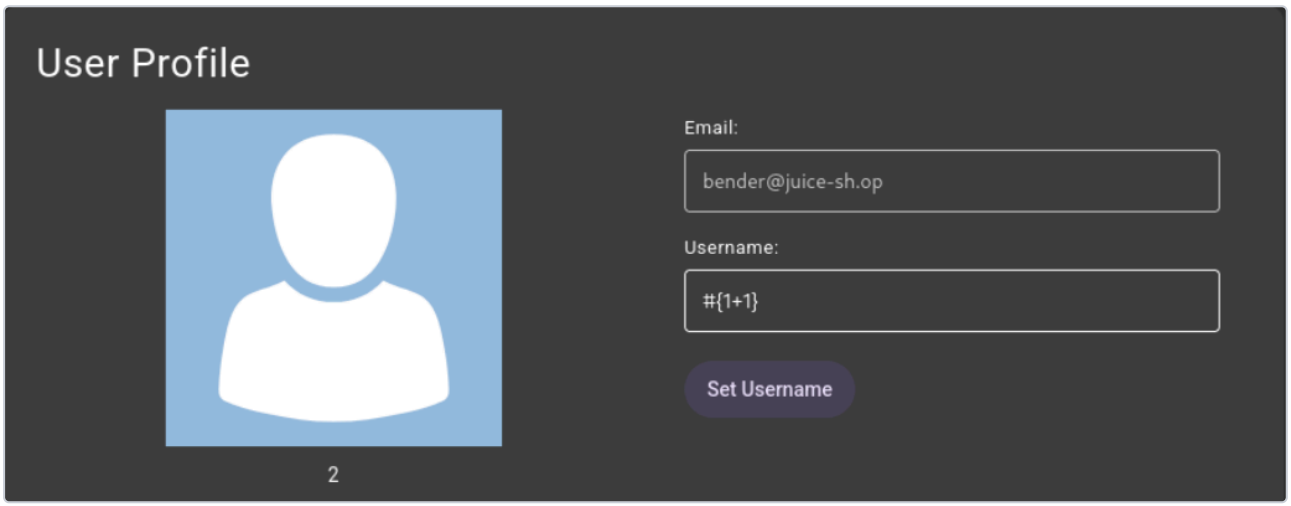



Figure 5. `#{1+1}` rendered as `2`, confirms Pug and server-side evaluation.

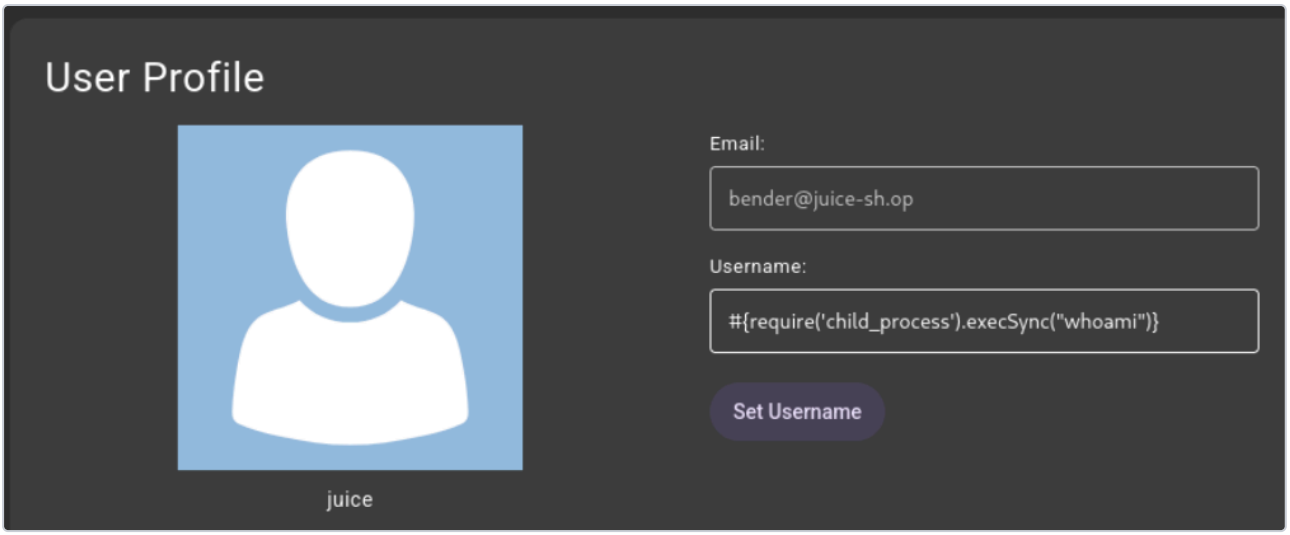



Figure 6. `whoami` returns `juice`.

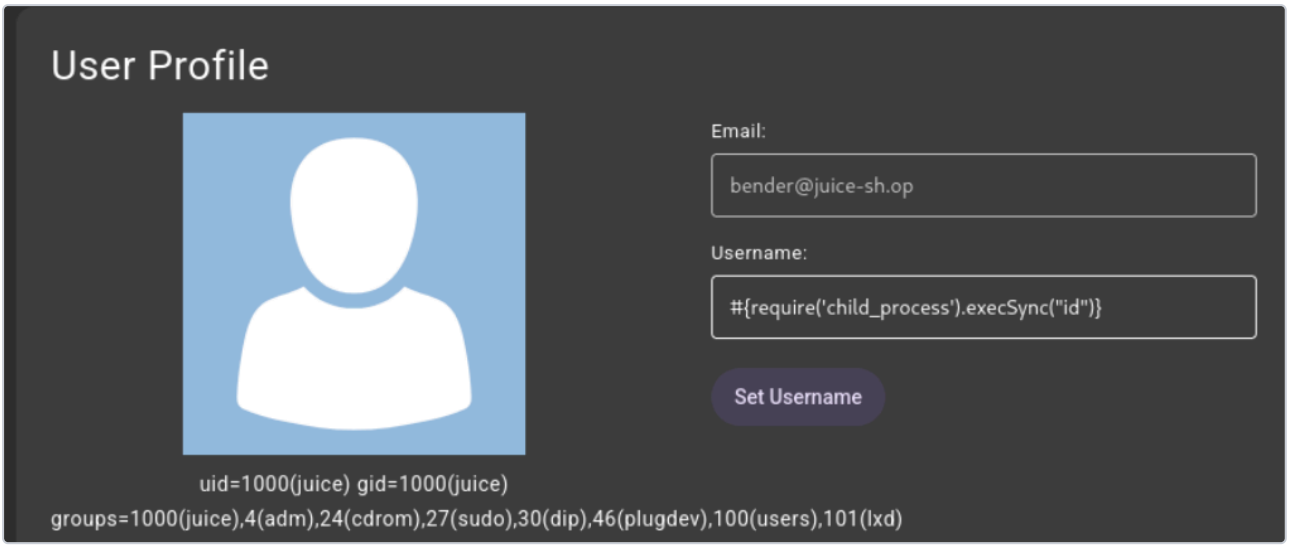



Figure 7. `id` returns `uid=1000(juice)` with `sudo` group.

**Impact**

Confirmed arbitrary command execution on the host as the `juice` service account. In this state an attacker can read and write files accessible to that account, read application secrets and the embedded database, and run further commands. The account's membership of the `sudo` group indicates a plausible privilege-escalation path (ATT&CK T1548.003, Sudo and Sudo Caching), although privilege escalation was not demonstrated in this assessment. Interactive shell access was not achieved in the lab, so the demonstrated impact is discrete command execution rather than a full interactive foothold. Because exploitation of the web application results in command execution on the host operating system, a component beyond the application's own security authority, the CVSS scope is assessed as Changed. Confidentiality, integrity, and availability impacts are all High.

**Remediation**

- **Primary fix:** Do not allow user input to be evaluated as template code. Pass the username to the template as a bound variable so the engine escapes it and treats it as data, rather than incorporating it into the evaluated template.
- Remove any path that extracts and evaluates the `#{ }` content of the username.

- Validate the username on input (an allow-list of permitted characters and a length limit) as defence in depth.
- Where server-side rendering of user content is genuinely required, use contextual auto-escaping and never raw evaluation of user-supplied strings.

**Verification of fix**

Re-run the identification probe. A patched application renders `#{1+1}` as the literal string rather than `2`, confirming the input is no longer evaluated.

---

## 4. Detection

This section describes how the attack can be detected generically, mapped to MITRE ATT&CK. The signals fall into three layers, strongest first.

- **Process lineage (highest fidelity).** The application runtime (the Node.js process serving Juice Shop) should never spawn a shell or system utility under normal operation. A `node` process spawning `sh`, `bash`, `id`, `whoami`, or similar is a high-confidence indicator of command execution through an injection flaw. Detection matches on parent image being the application process and child image being an interpreter or common utility. Maps to ATT&CK T1059 (.004 Unix Shell, .007 JavaScript).
- **Application and log analysis.** The attack leaves a recognisable pattern in request logs: a `POST /profile` whose username parameter contains template or runtime indicators (`#{`, `require(`, `child_process`, `process.mainModule`), typically followed by a `GET /profile` from the same session. Correlating these, and watching for `500` responses on `/profile` after a suspicious update, surfaces exploitation attempts. This layer is evadable through obfuscation and is best treated as corroborating evidence rather than a sole control. Maps to ATT&CK T1190.
- **Host artefacts.** Where a payload writes to disk or opens a network connection, file-integrity and egress monitoring on the host attribute the activity to the application process tree. Discovery commands such as `id` and `whoami` executed by the application process map to ATT&CK T1033.

Of these, the process-lineage signal is the most reliable, because it fires at the moment of execution and does not depend on recognising a specific payload string. The input and log signals are useful for attribution and for catching attempts, but should not be relied on alone given how easily payloads can be obfuscated.

---

## Appendices

**Appendix A: Lab environment**

| Component | Detail |
|---|---|
| Target | OWASP Juice Shop, `192.168.100.51:3000`, Ubuntu Server VM |
| Attacker | Kali Linux VM |
| Network | Host-only (isolated) |

| Component | Detail |
| --- | --- |
| Tooling | Burp Suite, Wappalyzer, browser |

**Appendix B: Payloads used**

| Purpose | Payload | Result |
| --- | --- | --- |
| Rule out `{{ }}` engines | `{{1+1}}` | Rendered literally (`\{{1+1}}`) |
| Confirm SSTI / identify Pug | `#{1+1}` | Rendered as `2` |
| Confirm command execution | `#{require('child_process').execSync("whoami")}` | `juice` |
| Confirm command execution and context | `#{require('child_process').execSync("id")}` | `uid=1000(juice) gid=1000(juice) groups=...,sudo,...` |

**Appendix C: Affected source files**

- `routes/userProfile.ts`: renders the profile and evaluates the stored username through Pug.
- `routes/updateUserProfile.ts`: updates the username from `req.body.username` with no validation.
- `views/userProfile.pug`: the profile template.

---

## References


1. OWASP Juice Shop Project: https://owasp.org/www-project-juice-shop/
2. PortSwigger: Server-Side Template Injection (Web Security Academy): https://portswigger.net/web-security/server-side-template-injection
3. PortSwigger Research: Server-Side Template Injection (James Kettle): https://portswigger.net/research/server-side-template-injection
4. PayloadsAllTheThings: SSTI (JavaScript / Pug): https://swisskyrepo.github.io/PayloadsAllTheThings/Server%20Side%20Template%20Injection/
5. MITRE ATT&CK: T1190 Exploit Public-Facing Application: https://attack.mitre.org/techniques/T1190/
6. MITRE ATT&CK: T1059 Command and Scripting Interpreter: https://attack.mitre.org/techniques/T1059/
7. MITRE ATT&CK: T1033 System Owner/User Discovery: https://attack.mitre.org/techniques/T1033/
8. CWE-1336: Improper Neutralization of Special Elements Used in a Template Engine: https://cwe.mitre.org/data/definitions/1336.html
9. FIRST: CVSS v3.0 Calculator: https://www.first.org/cvss/calculator/3.0